PROJECT APEX

Full Codebase Audit Report

Bible v5 vs. Actual Implementation

Generated March 09, 2026

Prepared by: Claude (Anthropic) — Automated Code Audit

Executive Summary

This report presents a comprehensive audit of the Project Apex codebase against the canonical engineering specification (Bible v5). Every source file in the project-apex folder was reviewed in full. The audit identifies: (1) architectural deviations from the Bible spec, (2) concrete code bugs that will cause incorrect behavior or runtime failures, (3) incomplete or stub implementations that will silently produce wrong results, and (4) pre-run concerns that must be addressed before any training run is executed.

Architecture Deviations from Bible	5	**Needs Resolution**
Critical Code Bugs (will cause errors / wrong results)	5	**Must Fix Before Run**
Incomplete / Stub Implementations	5	**Needs Attention**
Minor Issues / Documentation / Style	7	**Low Priority**
Pre-Run Blockers	3	**Must Resolve First**

Top 3 Must-Fix Before Any Training Run:

1. **Sector ID type mismatch:** Raw GICS sector codes (10, 15, 20...) are passed directly to nn.Embedding, which will crash or silently produce garbage embeddings.

2. **Off-by-one observation window:** _reconstruct_obs in sac_trainer.py returns L+1=61 timesteps when the model spec calls for L=60.

3. **Portfolio state stub:** 8 of the 20 global context features are permanently zero throughout training, meaning the agent cannot learn from its own portfolio history.

1. Files Reviewed

The following modules were read in their entirety and cross-referenced with Bible v5:

File	Description
`model/apex_actor_critic.py`	Full model architecture — actor, twin critics, attention stacks
`model/tcn.py`	Causal TCN encoder with dilated residual blocks
`model/attention.py`	Cross-asset multi-head self-attention module
`training/sac_trainer.py`	SAC update loop — critic, actor, alpha steps, polyak update
`training/replay_buffer.py`	Circular replay buffer with n-step staging, recency weights
`environment/trading_env.py`	Weekly trading environment — step, observation, cost model
`environment/reward_fn.py`	5-term shaped reward with cold-start blending
`environment/constraint_projector.py`	Dykstra projection onto feasibility set
`environment/market_data.py`	Market data cache builder — price arrays, sector IDs, rolling stats
`features/per_asset_features.py`	17 per-asset time-series features (§3.1)
`features/cross_sectional_features.py`	8 cross-sectional features (§3.2)
`features/macro_broadcast_features.py`	9 macro/global broadcast features (§3.3)
`features/benchmark_features.py`	3 QQQ benchmark features (§3.5)
`features/portfolio_state_features.py`	8 portfolio-state features stub (§3.4)
`features/normalizers.py`	Causal per-asset and fixed-scale normalizers (§3.6)
`features/feature_panel.py`	Full panel assembler [T, K_max, F] + global [T, D_g]
`config/master_config.yaml`	Master YAML configuration — all hyperparameters
`config/config_schema.py`	Dataclass schema with validation and cross-checks
`config/config_loader.py`	YAML loader with validation error handling
`evaluation/metrics.py`	Primary, secondary, tertiary OOS metrics
`apex_logging/apex_logger.py`	Structured 7-category logger + regression alarms
`integration/e2e_runner.py`	End-to-end integration test runner (Phase 12)

2. Architecture Deviations from Bible v5

These are cases where the implemented code differs from what Bible v5 specifies architecturally. Some deviations are deliberate and defensible design choices; others are inconsistencies that should be reconciled before training.

A1 — CrossAssetAttention: Separate Stacks vs. Shared Weights

Severity: HIGH — Significant parameter count and representational divergence

Bible §7.2: CrossAssetAttention is listed as "Shared" — implying the actor and both critics use the same attention weights to process cross-asset relationships.

Code (apex_actor_critic.py, attention.py): Three completely independent attention stacks are instantiated: `actor_attn`, `q1_attn`, and `q2_attn`. The attention.py docstring explicitly acknowledges this: "Three independent stacks are instantiated in ApexActorCritic: actor_attn, q1_attn, q2_attn (same architecture, separate parameters)."

Impact:

- Approximately 3x the attention layer parameter count vs. the shared design.
- The actor and critics learn different cross-asset representations, which breaks the implicit assumption in SAC that the critic and actor perceive the market structure similarly.
- Shared attention would allow gradient flow from both critic losses and the actor loss to jointly shape the cross-asset representation, improving sample efficiency.

Recommendation: Decide explicitly whether to keep separate stacks or revert to shared. If keeping separate, update the Bible §7.2 table to reflect the actual architecture. If reverting to shared, instantiate a single `CrossAssetAttention` and pass it to both the actor head and critic heads, taking care to handle the encoder gradient correctly (critic loss should flow through shared attention; actor loss should also flow through it via the encoder path).

A2 — F=25 vs. Bible §0.2 F=26

Severity: LOW — Internal Bible inconsistency; code is likely correct

Bible §0.2 hyperparameter table lists F=26 (features per asset). However, Bible §3.1.1 enumerates exactly 17 time-series features, and §3.2.1 enumerates exactly 8 cross-sectional features, totaling 25. The code (config, apex_actor_critic.py, feature_panel.py) consistently uses F=25, which matches the actual feature count from §3.1.1 and §3.2.1.

Recommendation: Correct the Bible §0.2 table entry from F=26 to F=25. The code is correct as-is. No code changes needed.

A3 — ret_12w vs. ret_13w (Naming and Window Size Discrepancy)

Severity: MEDIUM — Feature definition and naming differs from spec

Bible §3.1.1 lists feature `ret_13w` (13-week return). The code uses `ret_12w` with `DAYS_12W=60` (60 trading days = 12 weeks). This is both a naming discrepancy AND a window size difference: 13 weeks = 65 trading days, 12 weeks = 60 trading days. The cross-sectional feature module also references `ret_12w` when computing `ret_z_12w`. The QQQ benchmark feature `qqq_ret_12w` uses 60 days (12 weeks) and is consistent with the code, but the per-asset spec says 13 weeks.

Recommendation: Decide on one standard. If adopting 12 weeks (60 days) throughout, update Bible §3.1.1 to say `ret_12w`. If keeping 13 weeks (65 days) per the Bible, update DAYS_12W to 65 in per_asset_features.py and rename `ret_12w` to `ret_13w` everywhere. Both choices are defensible — 13 weeks is a calendar quarter, which has conceptual appeal.

A4 — Log-Sigma Bounds and Initialization

Severity: MEDIUM — Different initial exploration level and noise range

Bible §4.4 specifies: log_std range = [-5, 1], initialization = -1.5 (corresponding to sigma ≈ 0.22). **Code (apex_actor_critic.py)**: `log_sigma_min=-3.0`, `log_sigma_max=0.5`, `log_sigma_init=-1.0` (sigma ≈ 0.37). Both the initialization and the clamp bounds are different.

Impact: The agent starts with higher action noise (sigma 0.37 vs 0.22), which means more exploration early on. The narrower upper bound (0.5 vs 1.0) limits maximum noise. The tighter lower bound (-3.0 vs -5.0) prevents the agent from becoming fully deterministic. These are meaningful differences that affect the exploration-exploitation tradeoff, particularly in early training.

Recommendation: Align with the Bible spec (init=-1.5, range=[-5, 1]) unless there is a deliberate reason to deviate. Update apex_actor_critic.py defaults accordingly. Also update master_config.yaml to expose these as configurable parameters if they are not already.

A5 — Config Bible Version Mismatch

Severity: LOW — Documentation issue, no functional impact

Both `master_config.yaml` and the default in `config_schema.py` `MetadataConfig` show `bible_version: "v3"`. The project is built from Bible v5. This means any automated tooling or documentation that checks the bible_version field will report the wrong version.

Recommendation: Update `bible_version` to "v5" in both master_config.yaml and config_schema.py. Consider also bumping `config_version` from "2.0" to "2.5" or "3.0" to reflect that the config has been reconciled with Bible v5.

3. Code Bugs

These are bugs in the code that will cause runtime errors, silently incorrect results, or meaningfully wrong behavior. They must be fixed before any training run.

B1 — Sector ID Type Mismatch: Raw GICS Codes vs. Embedding Indices [CRITICAL]

Severity: CRITICAL — Will crash or produce incorrect embeddings at runtime

Location: `environment/market_data.py _build_sector_ids()` and `model/apex_actor_critic.py`

The Bug: `_build_sector_ids()` stores actual GICS sector codes (e.g., 10 for Energy, 15 for Materials, 20 for Industrials, 25 for Consumer Discretionary, etc.) in the `sector_ids_arr` as raw integer codes. These raw codes are then passed directly to `model.sector_emb = nn.Embedding(num_sectors, D_sector)` as embedding indices. Since `num_sectors=16` and GICS codes go up to 50 (Financials=40, IT=45, Communication=50), this will either raise an `IndexError` or, if the embedding table happens to be large enough, silently use completely meaningless embedding slots.

Fix: Create a mapping from GICS sector codes to contiguous zero-indexed integers before passing to the model. The mapping should be consistent across all folds and stored in the config or as a constant. Example:

```
GICS_TO_IDX = {10: 0, 15: 1, 20: 2, 25: 3, 30: 4, 35: 5, 40: 6, 45: 7, 50: 8, 55: 9, 60: 10, -1: 11} # -1 for inactive/unknown
```

Apply this mapping in `_build_sector_ids()` and make sure `num_sectors` in the model config matches the number of unique GICS codes plus the unknown slot (currently config has num_sectors not explicitly set, defaulting to whatever is passed at model instantiation). The e2e_runner.py uses sector_ids cycling 0..7 synthetically, which does work correctly — the real pipeline does not.

B2 — Off-by-One Observation Window in SACTrainer [CRITICAL]

Severity: CRITICAL — Model always receives 61 timesteps instead of the specified 60

Location: `training/sac_trainer.py _reconstruct_obs()`

The Bug: The function computes `win_len = self.L_lookback + 1`, then slices `panel_x[t-L:t+1]`, producing a window of L+1=61 timesteps. The model docstring specifies `x: [B, L, K, F]` with L=60. While the TCN receptive field (RF=63) technically accommodates 61 steps, this is an inconsistency with the spec and means every observation includes the current timestep t as the final element, plus 60 historical steps. Whether t is included in the spec is ambiguous, but the current window of 61 is one step wider than expected and will cause shape mismatches if any downstream code assumes L=60 exactly.

Fix: Decide whether L represents lookback-only (60 past steps, not including t) or lookback-plus-current (60 steps including t). If lookback-only, use `win_len = self.L_lookback` and slice `panel_x[t-L+1:t+1]`. If lookback-plus-current, update the Bible spec to say L=61 and update model documentation. Pick one convention and apply it consistently everywhere.

B3 — Dead Conditional in Trading Environment

Severity: MEDIUM — Silent no-op that obscures intent and may mask a real bug

Location: `environment/trading_env.py`, approximately line 301

The Bug: The conditional `obs = self._get_obs(t_next) if not done else self._get_obs(t_next)` has identical branches — both sides call `self._get_obs(t_next)`. This ternary is completely inert and was likely intended to return an empty observation (or zeros) when the episode is done, but the else-branch was inadvertently copied from the if-branch.

Fix: Determine the intended behavior when `done=True`. The standard SAC convention is that the terminal observation is not used in training (only the done flag matters). A common fix is:

```
obs = {} if done else self._get_obs(t_next)
```

Or alternatively, build the terminal observation but flag it as unused. Either way, the two branches must differ.

B4 — Double SiLU Activation in TCN Residual Block

Severity: LOW-MEDIUM — Extra nonlinearity not in spec; may affect training dynamics

Location: `model/tcn.py _TCNResBlock.forward()`

The Issue: Bible §7.4 specifies the residual block as: LayerNorm → CausalConv → SiLU → CausalConv → + skip. The code applies SiLU after conv1 (correct) but then applies a second SiLU to the final residual sum: `return F.silu(h + self.skip(x))`. This extra post-residual SiLU is not in the Bible spec. While post-residual activations exist in some architectures, they are not standard for Pre-LN residual blocks and may limit the gradient flow through the skip connection.

Fix: Change the final line to `return h + self.skip(x)` to match the Bible spec exactly. If the extra SiLU is intentional, document it explicitly and update Bible §7.4.

B5 — ret_rank_4w is Double Z-Scored in Cross-Sectional Features

Severity: MEDIUM — Feature meaning changes; rank becomes z-score of ranks

Location: `features/cross_sectional_features.py`, line 128

The Bug: Bible §3.2.1 defines `ret_rank_4w` as "Percentile rank of 4-week return." The code computes: `ret_rank_4w = _cs_zscore(_cs_rank(ret_4w_vals))`. `_cs_rank()` correctly returns percentile rank in [0, 1]. But then `_cs_zscore()` is applied on top, converting the rank values into z-scores of the ranks. This means `ret_rank_4w` is actually a z-score of percentile ranks, not a percentile rank. The output is no longer in [0, 1] and does not match the spec.

Fix: Change line 128 to simply: `ret_rank_4w = _cs_rank(ret_4w_vals)`. This preserves the percentile rank in [0, 1] as specified. The normalization/clipping in the normalizers.py pipeline will handle the final range transformation.

4. Incomplete and Stub Implementations

These are areas where code exists but functionality is not yet implemented, resulting in zeros, placeholders, or silent no-ops during training.

C1 — Portfolio State Features Are Permanently Zero [HIGH IMPACT]

Severity: HIGH — 8 of 20 global context features carry no information

Location: `features/portfolio_state_features.py`, `features/feature_panel.py`

The Issue: Bible §3.4.1 defines 8 portfolio-state features that form part of the global context vector g_t: turnover_last_step, realized_port_vol, current_drawdown, gross_exposure, rolling_excess_ret_qqq, estimated_cost_next_step, effective_n_positions, and market_vol_regime. These require the environment loop to compute properly. `compute_portfolio_state_stub()` returns `np.zeros(8)` for all of these, always. The feature panel calls this stub unconditionally. The model therefore sees g_t with 8 zero slots throughout all of training and OOS evaluation — the agent cannot learn to condition its behavior on its own portfolio state.

Impact: The agent will learn without knowing its current drawdown, its realized vol, its turnover, or whether it is overperforming/underperforming QQQ. These are critical signals for risk management behavior. Training without them is likely to produce a policy that is overconfident during high-drawdown regimes.

Recommendation: Implement real portfolio state computation in Phase 5 before any full training run. At minimum, the 8 features should be computed from the environment step() outputs (NAV history, w_exec history, cost history) and fed back into the panel. The stub is acceptable for Phase 1-4 unit tests only.

C2 — Missingness Tracking Is Incomplete

Severity: MEDIUM — Missing asset handling per §5.7 not fully implemented

Location: `environment/trading_env.py`

The Issue: Bible §5.7 specifies two missingness thresholds: (1) `missing_L[i]` — count of missing days within the L-day lookback window, with threshold=2, and (2) `streak_missing[i]` — consecutive missing days, with threshold=3. When either threshold is exceeded, the asset is flagged and treated as inactive. The trading_env.py only tracks mask transitions (new entries/exits) but does not maintain rolling windows of missingness counts or consecutive streaks. MissingnessConfig exists in config_schema.py with the correct thresholds, but no code reads these thresholds during the step() loop.

Recommendation: Implement the two missingness counters in `TradingEnv.__init__` and update them each step. Add the forced-inactive logic when either threshold is exceeded.

C3 — Only 2 of 8 Fold Panels Are Precomputed

Severity: HIGH — Training folds 3-8 cannot run without panel generation

Location: `Ticker_Data/panels_v2/`

The Issue: The walk-forward evaluation framework requires 8 folds. Panels exist only for fold_1 and fold_2. Folds 3 through 8 have no precomputed panel data. The SACTrainer loads panel data at initialization from these directories. Attempting to run training on folds 3-8 will either crash with a FileNotFoundError or produce incorrect results if a fallback is silently triggered.

Recommendation: Run the panel generation pipeline for all 8 folds before starting any training. Verify that the generation script covers all fold date ranges and that the output files match the expected naming convention (`panels_v2/fold_N/`).

C4 — Missing Phase 2 Gate Test

Severity: LOW — Phase 2 (Data Pipeline) has no automated gate test

All other phases have gate tests: test_phase1_gate1.py through test_phase13_gate13.py, plus test_data_validation.py and test_macro_evaluation_suite.py. `test_phase2_gate2.py` is absent. Bible §11.1 specifies 8 unit tests; the data pipeline (Phase 2) gates are not covered.

Recommendation: Create test_phase2_gate2.py covering: parquet file schema validation, data range checks, missing ticker handling, adj_factor non-negativity, and QQQ/VIX data completeness.

C5 — Outdated 'Stub' Comment in trading_env.py Reward

Severity: INFO — Documentation stale; reward_fn.py is fully implemented

The `TradingEnv` docstring says: "Reward is a stub: step() returns a dict of raw components. Full reward formula (§6) is wired in Phase 6." However, `reward_fn.py` exists and is fully implemented with all 5 reward terms, cold-start blending, and clipping per Bible §6. This comment is stale from an earlier development phase.

Recommendation: Update the TradingEnv docstring to reflect that the reward function is now fully implemented via reward_fn.py.

5. Minor Issues, Documentation, and Style

ID	Location	Issue	Recommendation
M1	environment/ market_data.py	The main build_market_data loop iterates over T*K_max slots in pure Python (~550,000 iterations for T=5000, K=110). This will be extremely slow — potentially taking minutes to hours to build the market data cache.	Vectorize the slot-filling loop using pandas pivot operations or NumPy advanced indexing instead of the nested Python for loop.
M2	environment/ market_data.py	_build_sector_ids() iterates through all snapshot dates and always overwrites sector assignments with the most recent value. This means historical GICS sector codes are not preserved — companies that changed sectors will have wrong historical assignments.	Build a time-indexed sector mapping: for each (date, security_id), use the most recent sector assignment on or before that date (as-of rule), not the most recent overall.
M3	config/ config_schema.py	ArchitectureConfig.L is documented as 'Lookback in weeks (§0.1 notation)' but L=60 is in trading DAYS (60 days = 12 weeks). The comment is incorrect and will confuse anyone reading the config.	Update comment to: '# Lookback window in trading days (L=60 = 12 weeks)'.
M4	training/ replay_buffer.py	buffer_reward_std uses np.std with ddof=0 (population std). ddof=1 (sample std) is more standard for variance estimation from a finite sample.	Change to np.std(rewards, ddof=1) in the buffer_reward_std property.
M5	integration/ e2e_runner.py	make_synthetic_market_data() hardcodes F=25 (x_panel shape). If F is changed experimentally, synthetic tests will break.	Add an F parameter to the function and pass it through from E2ERunner.
M6	evaluation/ metrics.py	information_ratio is computed identically to sharpe using compute_sharpe(). While mathematically correct (IR = Sharpe of excess returns when benchmark=QQQ), having the same underlying function for two named metrics is confusing.	Add a brief inline comment: '# IR = Sharpe of active returns; same formula when benchmark is QQQ'.
M7	config/ config_schema.py	The cross-validation assertion in ProjectConfig.validate() enforces embargo_weeks == n_step. This is correct per Bible §9.1 but the error message doesn't explain WHY (to prevent n-step returns from crossing fold boundaries).	Improve the assertion message to include the rationale: 'embargo_weeks must equal n_step to prevent n-step returns from crossing fold boundaries (§9.1)'.

6. Pre-Run Checklist

Before executing any training run, each of the following items must be addressed. Items marked BLOCKING will cause a crash or produce garbage results if not resolved. Items marked IMPORTANT will silently degrade model quality.

#	Item	Priority	Status
1	Fix B1: Map raw GICS sector codes to contiguous embedding indices before passing to nn.Embedding.	**BLOCKING**	Open
2	Fix B2: Resolve the L+1 vs L timestep ambiguity in _reconstruct_obs. Decide on the convention and apply consistently.	**BLOCKING**	Open
3	Implement C1: Portfolio state features. At minimum, compute turnover, running portfolio vol, and current drawdown from environment outputs. The full 8-feature vector per §3.4.1 is needed for complete training.	**IMPORTANT**	Open (stub)
4	Generate panels for folds 3-8 (C3). Verify output paths match expected naming convention.	**BLOCKING**	Open
5	Fix B3: Resolve dead conditional in trading_env.py for the done=True observation branch.	**IMPORTANT**	Open
6	Fix B5: Remove the double z-scoring in ret_rank_4w. This changes the meaning of that feature.	**IMPORTANT**	Open
7	Fix B4: Remove post-residual SiLU in _TCNResBlock to match Bible §7.4 spec.	MODERATE	Open
8	Resolve A1: Document final decision on shared vs. separate CrossAssetAttention stacks. Update Bible if separate is intentional.	**IMPORTANT**	Open
9	Resolve A3: Standardize on ret_12w (60 days) or ret_13w (65 days) throughout all feature modules.	MODERATE	Open
10	Resolve A4: Align log-sigma init and bounds with Bible §4.4 (init=-1.5, range=[-5, 1]).	MODERATE	Open
11	Update bible_version to 'v5' in config (A5).	LOW	Open
12	Vectorize the market_data.py slot-filling loop (M1) to prevent multi-hour build times.	**IMPORTANT**	Open
13	Fix sector history tracking in _build_sector_ids (M2) to use as-of rule per date.	MODERATE	Open
14	Create test_phase2_gate2.py for data pipeline validation (C4).	LOW	Open
15	Implement C2: full missingness tracking counters in TradingEnv per §5.7.	LOW	Open

7. What Is Working Well

Not everything needs fixing. The following areas are well-implemented, match the Bible spec cleanly, and require no changes:

`reward_fn.py`

5-term shaped reward is fully implemented and matches Bible §6 exactly: excess return / sigma_mkt, realized vol penalty, tail-risk squared penalty, cost penalty, constraint violation penalty — plus correct cold-start blending and ±5 clipping.

`constraint_projector.py`

Dykstra's alternating projections over the 3 constraint sets (long-only, full-investment, per-name cap) is correctly implemented, differentiable via autograd, and includes a correct equal-weight fallback on infeasibility.

`replay_buffer.py`

N-step return staging, recency weighting (half-life exponential), warmup episode handling (including correct last n_step-1 transition dropping), and the warmup exclusion threshold are all correctly implemented.

`normalizers.py`

Both CausalPerAssetNormalizer (rolling window z-score) and FixedScaleNormalizer (macro features) are correctly implemented with proper causal constraints. Final ±4.0 clipping is correctly applied.

`apex_logger.py`

Comprehensive 7-category structured logging, all 7 regression alarm types (including CRITICAL: mask_leak, nan_in_obs, constraint_violation), all 8 required OOS plots, per-fold and cross-fold summaries are all implemented.

`evaluation/metrics.py`

All 3 primary, 4 secondary, and 7 tertiary OOS metrics are correctly implemented including Excess CAGR, Sortino, Max Drawdown, CVaR(5%), Rank IC, and beta-to-QQQ.

`config_schema.py`

Cross-parameter validation (embargo_weeks == n_step, F == len(TS) + len(CS), TCN receptive field >= L, attn_d_model divisible by attn_num_heads) is thorough and will catch bad configs at load time.

`macro_broadcast_features.py`

Flexible column detection handles naming variations in macro_features.parquet gracefully. VIX change is correctly computed as a 5-day diff (not 1-day as stored in parquet) per Bible §M4. HYG pre-launch imputation is handled.

`feature_panel.py`

_build_asof_membership correctly implements the backward-fill only as-of rule for NDX membership, preventing look-ahead bias in the universe.

`e2e_runner.py`

Integration test framework covers all 3 required tests: E2E episode, 500-step SAC update, and determinism check per Bible §11.2.

8. Prioritized Fix Summary

The table below consolidates all findings in priority order. Fix the CRITICAL items before any test run; HIGH items before the first full training fold.

CRITICAL	B1	Sector GICS codes not mapped to embedding indices — will crash	`market_data.py`	1-2h
CRITICAL	C3	Only 2/8 fold panels precomputed — folds 3-8 cannot run	`Ticker_Data/`	Pipeline run
CRITICAL	B2	_reconstruct_obs returns L+1=61 timesteps; spec says L=60	`sac_trainer.py`	30 min
HIGH	C1	Portfolio state features permanently zero — agent blind to its own state	`portfolio_state_features.py`	1-2 days
HIGH	M1	market_data.py slot-filling loop is O(T*K) Python — extremely slow	`market_data.py`	2-4h
HIGH	B5	ret_rank_4w double z-scored — wrong feature meaning	`cross_sectional_features.py`	15 min
HIGH	B3	Dead conditional in trading_env.py done=True observation branch	`trading_env.py`	30 min
HIGH	A1	Three separate attention stacks vs. Bible's 'Shared' — must decide and document	`apex_actor_critic.py`	Design call
MEDIUM	B4	Extra SiLU at TCN residual output — not in Bible §7.4	`tcn.py`	5 min
MEDIUM	A4	Log-sigma init=-1.0, range=[-3,0.5] vs. Bible init=-1.5, range=[-5,1]	`apex_actor_critic.py`	30 min
MEDIUM	A3	ret_12w (60d) vs. Bible ret_13w (65d) — naming and window mismatch	`per_asset_features.py`	1-2h
MEDIUM	M2	Sector history uses most-recent code, not as-of rule	`market_data.py`	2h
MEDIUM	C2	Missingness threshold tracking incomplete per §5.7	`trading_env.py`	4h
LOW	A2	Bible §0.2 says F=26; code correctly uses F=25 — update Bible	`Bible §0.2`	Doc edit
LOW	A5	Config bible_version says 'v3'; should be 'v5'	`master_config.yaml`	5 min

LOW	C4	test_phase2_gate2.py missing — data pipeline has no gate test	`tests/`	4-8h
LOW	M3-M7	Minor doc, comment, and style issues	`Various`	< 1h total

9. Confirmed Correct Hyperparameters

The following Bible §0.2 hyperparameters were verified in the code and match the spec exactly. No changes needed.

Parameter	Bible §0.2 Value	Code Value	Location
gamma (discount)	0.975	0.975	SACConfig / sac_trainer.py
n_step (N-step returns)	4	4	SACConfig / replay_buffer.py
tau (Polyak)	0.005	0.005	SACConfig / sac_trainer.py
K_max (universe)	110	110	ArchitectureConfig
L (lookback window)	60	60	ArchitectureConfig
TCN channels	128	128	ArchitectureConfig / tcn.py
TCN levels	5	5	ArchitectureConfig / tcn.py
TCN kernel_size	3	3	ArchitectureConfig / tcn.py
TCN dilation_base	2	2	ArchitectureConfig / tcn.py
Attention heads	4	4	ArchitectureConfig / attention.py
Attention d_model	128	128	ArchitectureConfig / attention.py
Attention d_ff	256	256	ArchitectureConfig / attention.py
Attention layers	2	2	ArchitectureConfig / attention.py
D_ticker (embedding)	32	32	ArchitectureConfig
D_sector (embedding)	8	8	ArchitectureConfig
N_quantiles (critic)	32	32	ArchitectureConfig
per_name_cap	0.15	0.15	ConstraintConfig
sector_cap	0.35	0.35	ConstraintConfig
lambda_slow	0.75	0.75	RewardConfig
lambda_tail	0.40	0.40	RewardConfig
lambda_cost	1.0	1.0	RewardConfig
lambda_cv	1.0	1.0	RewardConfig
sigma_mkt window	13 weeks	13 weeks	RewardConfig
sigma_port window	52 weeks	52 weeks	RewardConfig
Replay buffer capacity	800	800	ReplayConfig
Warmup steps	52	52	ReplayConfig
Warmup excl. threshold	128	128	ReplayConfig
Recency half-life	3.0 years	3.0 years	ReplayConfig
n_folds (walk-forward)	8	8	EvaluationConfig
embargo_weeks	4	4	EvaluationConfig
bootstrap_resamples	10000	10000	EvaluationConfig
critic_lr	3e-4	3e-4	OptimizerConfig
actor_lr	1e-4	1e-4	OptimizerConfig
encoder_lr	3e-4	3e-4	OptimizerConfig

updates_per_step	20	20	SACConfig
policy_delay	2	2	SACConfig
batch_size	64	64	SACConfig

End of Audit Report

Project Apex | Bible v5 Compliance Audit | March 09, 2026

Summary: 22 files reviewed. 5 architecture deviations, 5 code bugs, 5 incomplete implementations, 7 minor issues. 3 blocking pre-run items require immediate resolution (sector embedding mismatch, missing fold panels, off-by-one observation window). The core SAC algorithm, constraint projector, reward function, replay buffer, evaluation metrics, and logging infrastructure are all correctly implemented and match the Bible spec.